

Trabajo 4 de Bases de Datos 2

1. Introducción

El objetivo de este trabajo es resolver y comparar varias formas de una misma consulta SQL: obtener todas las parejas de proveedores que venden exactamente el mismo conjunto de productos. La solución se desarrolla en Oracle y PostgreSQL, usando las mismas tablas y muestras de datos comparables.

El trabajo se hizo con Docker para que las pruebas puedan repetirse sin instalar manualmente los motores de base de datos en el sistema operativo.

2. Objetivos

- Crear las tablas `Prov` y `ProvxBProd` respetando la estructura del enunciado.
- Escribir cuatro consultas SQL distintas para resolver el problema.
- Crear un programa PL/SQL en Oracle que cargue datos aleatorios consistentes.
- Ejecutar pruebas con tamaños de muestra crecientes.
- Obtener Explain Plan y TKPROF en Oracle.
- Obtener métricas equivalentes en PostgreSQL mediante `EXPLAIN ANALYZE`.
- Comparar el comportamiento de las consultas en ambos motores.

3. Modelo De Datos

La tabla `Prov` almacena proveedores:

```
1 Prov(nit, nombre)
```

La tabla `ProvxBProd` almacena los productos que vende cada proveedor:

```
1 ProvxBProd(nit, codigoProducto)
```

Restricciones:

- `Prov.nit` es clave primaria.
- `Prov.nombre` es único.
- `(ProvxBProd.nit, ProvxBProd.codigoProducto)` es clave primaria compuesta.
- `ProvxBProd.nit` es clave foránea hacia `Prov`.

No se agregaron columnas adicionales porque el enunciado indica que no debe modificarse la estructura.

4. Metodología

Las pruebas se ejecutan con Docker Compose. Se levantan dos contenedores: Oracle Free y PostgreSQL.

Para cada muestra se realiza este proceso:

1. Se eliminan los datos anteriores.
2. Se cargan proveedores.
3. Se cargan relaciones proveedor-producto aleatorias, evitando duplicados.
4. Se ejecutan las cuatro consultas.
5. En Oracle se genera Explain Plan.
6. En Oracle se activa traza SQL y se procesa con TKPROF.
7. En PostgreSQL se ejecuta `EXPLAIN (ANALYZE, BUFFERS)`.
8. Se guardan los resultados en archivos de texto.

Los tamaños de muestra propuestos son:

Muestra	Proveedores	Productos posibles	Filas en ProvxProd
1	100	1.000	1.000
2	500	5.000	5.000
3	1.000	10.000	10.000
4	2.000	20.000	25.000
5	5.000	50.000	50.000

5. Generación De Datos

En Oracle se usa el procedimiento `cargar_datos_aleatorios`, escrito en PL/SQL. Primero borra los datos anteriores, luego inserta proveedores con `nit` consecutivo y nombre único. Después genera pares aleatorios (`nit`, `codigoProducto`). Si un par ya existe, se ignora y se intenta generar otro.

La clave foránea siempre es consistente porque los valores de `nit` se generan entre 1 y el número de proveedores creados. También pueden quedar proveedores sin productos. Esto es correcto, porque el enunciado contempla el caso de proveedores con conjunto vacío de productos. (Los primeros 20 proveedores se dejan sin productos para garantizar los casos de conjunto vacío).

En PostgreSQL se implementa una versión equivalente en PL/pgSQL.

6. Explicación De Las Cuatro Consultas

6.1. Consulta 1: doble NOT EXISTS

Esta consulta compara cada pareja de proveedores (`p1`, `p2`). La condición exige dos cosas: que no exista un producto vendido por `p1` que no venda `p2`, y viceversa. Esta consulta es distinta porque usa subconsultas correlacionadas y expresa la igualdad como ausencia de diferencias (división relacional).

6.2. Consulta 2: agrupación y firma del conjunto

Esta consulta crea una representación textual del conjunto de productos de cada proveedor (firma). En Oracle usa `LISTAGG`; en PostgreSQL usa `STRING_AGG`. Los productos se ordenan antes de concatenarlos.

Nota Técnica Importante: En Oracle, el operador `LISTAGG` tiene un límite de 4000 bytes (o 32767 con tipos extendidos). Si un proveedor vendiera una cantidad masiva de productos, esto generaría el error `ORA-01489: result of string concatenation is too long`. Para este experimento,

el número de productos por proveedor no excede el límite, pero en producción esto representa un riesgo de escalabilidad frente a enfoques puramente relacionales (como Q1 o Q3).

6.3. Consulta 3: conteos y productos comunes

Esta consulta calcula cuántos productos vende cada proveedor y cuántos tienen en común. Si tienen la misma cantidad total y los comunes coinciden con el total, venden el mismo conjunto. Es distinta porque se basa en agregaciones numéricas y no en operadores de conjuntos.

6.4. Consulta 4: operadores de conjuntos

En Oracle se usa MINUS; en PostgreSQL se usa EXCEPT. Revisa que las diferencias de conjuntos entre ambos proveedores sean vacías bidireccionalmente.

7. Resultados y Análisis

Todas las consultas retornaron 190 filas en todas las muestras, ya que se forzaron 20 proveedores sin productos, resultando en $20 \times 19/2 = 190$ parejas que comparten el conjunto vacío.

7.1. Tablas de Resultados

7.1.1. Tabla Resumen Oracle - Explain Plan

Muestra	Consulta	Cost	Elapsed	Observaciones
100 / 1.000	Q1 - NOT EXISTS	4472	0.01s	MERGE JOIN + FILTER, INDEX RANGE SCAN en PK_PROVXPROD
100 / 1.000	Q2 - LISTAGG	218	0.00s	TEMP TABLE TRANSFORMATION, HASH JOIN a tabla temporal
100 / 1.000	Q3 - conteos	26	0.03s	TEMP TABLE TRANSFORMATION, HASH GROUP BY, NESTED LOOPS
100 / 1.000	Q4 - MINUS	3356	0.02s	MERGE JOIN + MINUS con INDEX RANGE SCAN
500 / 5.000	Q1 - NOT EXISTS	4472	0.21s	Mismo plan, costo no refleja el crecimiento de datos
500 / 5.000	Q2 - LISTAGG	220	0.00s	Plan estable, costo sube ligeramente
500 / 5.000	Q3 - conteos	30	0.71s	Mismo plan, NESTED LOOPS sobre 124.750 pares
500 / 5.000	Q4 - MINUS	3356	0.39s	Mismo plan, MINUS se ejecuta por cada par
1.000 / 10.000	Q1 - NOT EXISTS	4472	0.85s	Plan idéntico, tiempo real crece
1.000 / 10.000	Q2 - LISTAGG	224	0.00s	Plan estable y eficiente
1.000 / 10.000	Q3 - conteos	41	2.94s	Plan hash cambia a 2544319945 (full scan en Prov)
1.000 / 10.000	Q4 - MINUS	3356	1.58s	Mismo plan, 499.500 pares evaluados
2.000 / 25.000	Q1 - NOT EXISTS	4476	3.44s	Costo estimado apenas 4476, pero real 3.44s
2.000 / 25.000	Q2 - LISTAGG	234	0.02s	Plan estable, más rápido que Q1/Q3/Q4 por órdenes de magnitud
2.000 / 25.000	Q3 - conteos	63	16.40s	Usa disco (7335 lecturas físicas); alto costo real
2.000 / 25.000	Q4 - MINUS	3360	6.37s	1.999.000 pares, 4.78M de LIOs

Muestra	Consulta	LIOs / filas proc.	Filas ret. / fetches	Disco / LIOs
100 / 1.000	Q1	68.34	13.57	0.0000
100 / 1.000	Q2	0.09	13.57	0.0000
100 / 1.000	Q3	77.36	13.57	0.0000
100 / 1.000	Q4	69.43	13.57	0.0000
500 / 5.000	Q1	1,560.77	13.57	0.0000
500 / 5.000	Q2	0.12	13.57	0.0000
500 / 5.000	Q3	1,894.99	13.57	0.0000
500 / 5.000	Q4	1,563.44	13.57	0.0000
1.000 / 10.000	Q1	6,197.84	13.57	0.0000
1.000 / 10.000	Q2	0.20	13.57	0.0000
1.000 / 10.000	Q3	7,903.71	13.57	0.0000
1.000 / 10.000	Q4	6,201.65	13.57	0.0000
2.000 / 25.000	Q1	25,164.63	13.57	0.0000
2.000 / 25.000	Q2	0.41	13.57	0.0000
2.000 / 25.000	Q3	14,038.62	13.57	0.0027
2.000 / 25.000	Q4	25,193.05	13.57	0.0000

7.1.2. Tabla Resumen Oracle - Métricas (v\$sql)

7.1.3. Tabla Resumen PostgreSQL

Muestra	Consulta	Execution Time	shared hit	shared read	Observaciones
100 / 1.000	Q1	23.293 ms	220	0	Hash Anti Join, Nested Loop, Seq Scan + Index Scan
100 / 1.000	Q2	0.721 ms	9	0	Hash Join con CTE, GroupAggregate
100 / 1.000	Q3	10.954 ms	216	0	Hash Join con CTE, HashAggregate, Nested Loop
100 / 1.000	Q4	74.343 ms	69.881	0	Nested Loop, SubPlan con HashSetOp Except
500 / 5.000	Q1	60.035 ms	2.522	0	Mismo plan; 500 x 250 filas en Nested Loop
500 / 5.000	Q2	3.079 ms	34	0	Plan eficiente, firma por STRING_AGG
500 / 5.000	Q3	207.707 ms	2.499	0	Hash Right Join produce 1.2M filas intermedias
500 / 5.000	Q4	1.762.754 ms	2.824.693	0	SubPlan se ejecuta 124.750 veces; 2.8M buffers
1.000 / 10.000	Q1	208.851 ms	200.728	0	Nested Loop produce 499.500 filas internas
1.000 / 10.000	Q2	6.774 ms	85	0	Plan estable, 10.020 filas agrupadas
1.000 / 10.000	Q3	951.194 ms	199.433	temp: 1511	Usa disco por primera vez (hash batches: 4)
1.000 / 10.000	Q4	7.946.117 ms	15.862.912	0	499.500 iteraciones del SubPlan; 15.8M buffers
2.000 / 25.000	Q1	801.607 ms	398.426	0	1.999.000 filas en Nested Loop; JIT habilitado
2.000 / 25.000	Q2	17.639 ms	183	0	Plan cambia a Merge Join (antes Hash Join)
2.000 / 25.000	Q3	4.030.356 ms	396.940	temp: 10128	32 batches hash, 24.6M filas intermedias
2.000 / 25.000	Q4	19.167.849 ms	34.839.945	0	1.999.000 iteraciones, Index Only Scan, 34.8M buffers

7.2. Comparación de Rendimiento (Oracle vs PostgreSQL)

7.2.1. Tiempo de ejecución

La consulta más rápida fue Q2 (LISTAGG / STRING_AGG). Se mantuvo muy rápida (menor a 20 ms en Postgres y menor a 0.02s en Oracle) a pesar del crecimiento de la muestra. La consulta más lenta fue Q4 (MINUS / EXCEPT), llegando a 19.17s en PostgreSQL con 2.000 proveedores.

7.2.2. Lecturas lógicas y Planes de Ejecución

La enorme diferencia de rendimiento en Q4 (donde PostgreSQL procesó 34.8M de *shared hits* vs Oracle con 4.78M *LIOs*) radica en el plan de ejecución:

- **PostgreSQL** anida el **EXCEPT** en un *SubPlan* evaluado exhaustivamente por cada pareja.
- **Oracle** utiliza un **INDEX RANGE SCAN** dentro de la operación **MINUS**. Esto ocurre porque Oracle aprovecha el índice compuesto (B-Tree) generado automáticamente en la Llave Primaria **ProvxDProd(nit, codigoProducto)**. Al ser **nit** el primer atributo del índice, la base de datos puede resolver el **MINUS** mediante un *Index Only Scan*, sin necesidad de acceder a los bloques de la tabla.

8. Conclusiones

1. **La consulta más eficiente** fue Q2 (Firma con agregación de strings). Transformar un problema de conjuntos relacionales a un problema escalar evita el crecimiento cuadrático. Sin embargo, tiene la limitación de la longitud de bytes en Oracle.
2. **Operadores de conjuntos (MINUS / EXCEPT)**: Fueron los enfoques más intuitivos, pero los de peor rendimiento.
3. **El crecimiento cuadrático**: El problema base implica comparar combinaciones de N elementos ($N \times (N - 1)/2$). Q1, Q3 y Q4 escalan muy mal con grandes volúmenes.
4. **Oracle vs PostgreSQL**: Oracle supo aprovechar sus índices por defecto de manera más eficiente en los operadores **MINUS**, mientras que PostgreSQL fue superior en la manipulación rápida de **STRING_AGG**.